

Chapter Three

System Methodology and Architecture

3.1 Overview and Research Objectives

Autonomous navigation of Unmanned Aerial Vehicles (UAVs) in dense urban environments presents profound interdisciplinary challenges across robotics, computer vision, control theory, and high-performance simulation. Unlike open-field or rural flight corridors, metropolitan areas feature severe Global Navigation Satellite System (GNSS) degradation due to signal attenuation and multipath reflections caused by tall structures—commonly referred to as 'urban canyons'. Simultaneously, micro-UAVs must negotiate complex three-dimensional aerodynamic profiles, sudden crosswinds, tight spatial boundaries, and unexpected dynamic obstacles (such as construction cranes, utility poles, and moving infrastructure) while adhering to rigorous SWaP-C (Size, Weight, Power, and Cost) constraints.

To address these formidable challenges, this research proposes a comprehensive, closed-loop autonomous navigation methodology that harmoniously integrates four interconnected subsystems: (1) high-fidelity photorealistic three-dimensional urban simulation with physics engine and synthetic sensor generation; (2) a mathematically rigorous 15-state Error-State Extended Kalman Filter (ESKF) for optimal kinematic state estimation under multi-sensor fusion; (3) a learned temporal trajectory controller driven by a Gated Recurrent Unit (GRU v3) deep neural network; and (4) an onboard edge-computing vision model—PULP-DroNet v3—for real-time reactive collision avoidance and lateral visual servoing.

Primary Research Objective: Formulate, implement, and validate an integrated navigation architecture capable of guiding an autonomous quadrotor through GPS-degraded, obstacle-dense three-dimensional urban corridors without human

intervention, maintaining sub-metre trajectory tracking fidelity while reactively avoiding sudden obstacles at thirty or more frames per second.

3.2 Overall System Architecture and Microservice Topology

The system is designed around a decoupled, asynchronous microservices architecture that separates the heavy graphical and physical rendering loop from computationally intensive deep learning inference engines. In conventional monolithic simulation frameworks, running deep convolutional neural networks and recurrent models inside the main graphics loop causes severe frame drops, unstable delta-time physics integration, and thread locking. The proposed architecture decouples the system into three distinct microservice tiers operating over dedicated high-speed local TCP/IP communication channels:

Tier 1 — WebGL Simulation Client (Three.js / Vite, Port 3000): Executes the procedural three-dimensional urban synthesis, six-degree-of-freedom rigid-body quadrotor flight dynamics, synthetic sensor generation (IMU, GPS, Barometer, Magnetometer, Monocular Vision), real-time Error-State Kalman Filtering, and cinematic / onboard camera rendering.

Tier 2 — GRU Learned Navigation Service (FastAPI / PyTorch, Port 8765): Receives sliding temporal sequences of 24 kinematic and geometric features over 101 historical timesteps, providing smooth velocity correction vectors (Δv) that optimise global trajectory tracking and compensate for nonlinear aerodynamic disturbances.

Tier 3 — PULP-DroNet Obstacle Avoidance Service (FastAPI / PyTorch, Port 8766): Receives real-time 200×200 grayscale visual feeds from the UAV's forward-facing sensor camera, executing deep residual convolutional inference to output continuous steering commands and collision probability estimates in fewer than twelve milliseconds per frame.

Table 3.1: Functional Roles, Technology Stacks, and Interface Contracts of Each Microservice

Service	Port / Protocol	Technology Stack	Input $\rightarrow$ Output	Latency Target
Simulation Engine	3000 (HTTP/WS)	Three.js, Vite, WebGL 2	Controls $\rightarrow$ 3D Scene & Sensor Data	60 FPS (≤ 16.6 ms)

GRU Navigation AI	8765 (REST/JSON)	FastAPI, PyTorch, NumPy	$[101 \times 24]$ Features $\rightarrow \Delta v$ (vx, vy, vz)	< 15 ms
PULP-DroNet AI	8766 (REST)	FastAPI, PyTorch, Pillow	200×200 Image $\rightarrow$ Steering α , P_coll	< 12 ms

3.3 Simulation Environment and Three-Dimensional Urban Synthesis

A critical prerequisite for validating vision-based and neural-network navigation algorithms is a simulation environment that accurately reproduces the visual distribution and physical scale of modern metropolitan infrastructure. Low-fidelity block worlds suffer from an acute simulation-to-reality (sim-to-real) gap, causing deep visual models such as DroNet to fail completely when exposed to real-world textures, lighting variations, and geometric shadows. The simulation environment in this project combines procedural city synthesis with high-end Physically Based Rendering (PBR) and hardware-accelerated instanced three-dimensional asset streaming.

3.3.1 Procedural City Generation and Spatial Zoning

The procedural generation pipeline divides the world space into structured Cartesian grids comprising diverse urban zones: (1) a Commercial Central Business District (CBD) characterised by dense skyscrapers ranging from 60 to 180 metres in height; (2) a mid-rise Residential Sector featuring brick, plaster, and concrete apartment blocks (20 to 45 metres); (3) an Industrial Zone with wide steel warehouses, oil tanks, and utility bridges; and (4) an Infrastructure and Transportation Network with multi-lane asphalt avenues, intersection crosswalks, sidewalk boundaries, and perimeter green spaces. Roads are mathematically indexed as spatial flight corridors with explicit waypoint sequences that allow the autonomous route controller to simulate corridor-constrained urban drone delivery missions.

3.3.2 Physically Based Rendering (PBR) Material Architecture

To achieve visual realism without sacrificing real-time performance, the rendering engine utilises a standard Metallic-Roughness PBR material pipeline conforming to the microfacet Cook-Torrance specular reflectance model. Texture maps are loaded dynamically from the production asset library and assigned according to surface typology:

Diffuse / Albedo Map (sRGB Colour Space): Encodes the base surface colour free of directional lighting and shadow information. Stored in sRGB space (THREE.SRGBColorSpace) to guarantee correct gamma correction across display devices.

Normal Map (Linear Colour Space, Tangent-Space OpenGL): Encodes micro-surface perturbations—cracks in asphalt, mortar joints in brick, panel seams in concrete—using three-channel RGB vectors, enabling convincing specular highlights and surface relief without increasing vertex counts.

Roughness and Metalness Maps (Linear Colour Space): Define microfacet distribution and electrical conductivity respectively. Road surfaces exhibit high roughness with specular water puddle appearance, while glass and metallic facades feature high specular reflectivity.

Texture Wrapping and UV Repetition: Large structures utilise THREE.RepeatWrapping with proportional UV scaling factors calibrated to metric world coordinates (repeating road textures every 25 metres), eliminating both visual tiling patterns and blurred oversized textures.

3.3.3 Three-Dimensional GLB Asset Pipeline and Hardware-Accelerated Instancing

In addition to procedural geometry, the environment integrates a library of detailed three-dimensional assets in binary glTF format (.glb). These include realistic streetlights, traffic signals, road barriers, guardrails, bridges, rooftop air-conditioning chillers, satellite antennas, and multi-species urban vegetation. To prevent catastrophic GPU draw-call bottlenecks, repeated assets are rendered using THREE.InstancedMesh. A single GPU draw call renders thousands of trees and street props by feeding transformation matrices (position, rotation, scale) directly into vertex buffer attributes, maintaining sixty frames per second rendering speed across the full city environment.

3.4 State Estimation: 15-State Error-State Extended Kalman Filter (ESKF)

Accurate state estimation is the bedrock of autonomous flight. In real-world quadrotors, raw Inertial Measurement Unit (IMU) sensors suffer from stochastic thermal drift, bias instability, and high-frequency vibration noise. Integrating raw accelerometer and gyroscope signals directly causes estimated position to diverge quadratically within seconds—a phenomenon known as dead-reckoning drift. To resolve this fundamental limitation, the navigation stack formulates a 15-State Error-State Extended Kalman Filter (ESKF).

3.4.1 State Vector Parameterisation and Lie Group Formulation

In standard Extended Kalman Filters (EKFs), attitude orientation is either parameterised by Euler angles—which suffer from mathematical singularities known as gimbal lock—or by four-dimensional unit quaternions, which require over-parameterised covariance matrices and suffer from covariance rank deficiency on the unit sphere S^3 . The Error-State formulation solves this elegantly by splitting the true kinematic state into a large-signal nominal state propagated by IMU integration, and a small-signal error state corrected by sensor measurements:

$$\mathbf{x}_{\text{true}} = \mathbf{x}_{\text{nom}} \oplus \delta\mathbf{x} \quad (3.1)$$

The complete 15-dimensional state vector is partitioned as follows:

$$\mathbf{x}_{\text{nom}} = [\mathbf{p}, \mathbf{v}, \mathbf{q}, \mathbf{b}_a, \mathbf{b}_\omega]^T \in \mathbb{R}^3 \times \mathbb{R}^3 \times S^3 \times \mathbb{R}^3 \times \mathbb{R}^3 \quad (3.2)$$

$$\delta\mathbf{x} = [\delta\mathbf{p}, \delta\mathbf{v}, \delta\theta, \delta\mathbf{b}_a, \delta\mathbf{b}_\omega]^T \in \mathbb{R}^{15} \quad (3.3)$$

where $\mathbf{p} \in \mathbb{R}^3$ is three-dimensional position (m), $\mathbf{v} \in \mathbb{R}^3$ is velocity (m s^{-1}), $\mathbf{q} \in S^3$ is the unit quaternion encoding attitude, $\mathbf{b}_a \in \mathbb{R}^3$ is the accelerometer bias vector (m s^{-2}), $\mathbf{b}_\omega \in \mathbb{R}^3$ is the gyroscope bias vector (rad s^{-1}), and $\delta\theta \in \mathbb{R}^3$ is the minimal three-dimensional rotation vector in the tangent space of $\text{SO}(3)$ —avoiding both gimbal lock and rank deficiency. Because the error state $\delta\mathbf{x}$ always remains near zero between measurement updates, its dynamics are strictly linear, eliminating the higher-order linearisation errors that degrade conventional EKFs.

3.4.2 Continuous-Time Kinematics and IMU Propagation

Between sensor measurements, the nominal state is propagated forward at 100 Hz using raw IMU measurements—specific force a_m from the accelerometer and angular rate ω_m from the gyroscope—according to continuous-time rigid-body kinematics:

$$dp/dt = v \quad (3.4a)$$

$$dv/dt = R(q) \cdot (a_m - b_a) + g \quad (3.4b)$$

$$dq/dt = \frac{1}{2} \cdot q \otimes (\omega_m - b_\omega) \quad (3.4c)$$

$$db_a/dt = w_{ba} \quad (\text{accelerometer bias random walk}) \quad (3.4d)$$

$$db_\omega/dt = w_{b\omega} \quad (\text{gyroscope bias random walk}) \quad (3.4e)$$

Here $R(q) \in SO(3)$ is the rotation matrix derived from the nominal quaternion q , $g = [0, 0, -9.81]^T \text{ m s}^{-2}$ is the gravitational acceleration vector, and w_{ba} , $w_{b\omega}$ are zero-mean Gaussian process noise terms. Simultaneously, the 15×15 error covariance matrix P is propagated in discrete time:

$$P_{\{k|k-1\}} = \Phi_k \cdot P_{\{k-1\}} \cdot \Phi_k^T + Q_d \quad (3.5)$$

$$\Phi_k \approx I_{15} + F_x \cdot \Delta t \quad (3.6)$$

where F_x is the linearised error dynamics matrix capturing cross-coupling between error states—specifically, accelerometer errors couple into velocity through the rotation matrix $R(q)$, and attitude errors couple into velocity through the skew-symmetric matrix of the corrected specific force $[a_m - b_a] \times$. Q_d is the discrete-time process noise covariance derived from IMU noise spectral density parameters.

3.4.3 Multi-Sensor Measurement Update and Error Injection

When discrete sensor measurements arrive asynchronously, the filter executes a standard Kalman correction step. Three external sensors provide independent measurement updates:

GPS Position and Velocity (10 Hz): Updates δp (position error) and δv (velocity error) with measurement noise covariance R_{GPS} .

Barometric Altimeter (20 Hz): Updates the vertical position error component δp_z with high vertical precision ($\sigma_{\text{baro}} \approx 0.2 \text{ m}$), compensating for GPS vertical inaccuracy.

Three-Axis Magnetometer (25 Hz): Corrects heading yaw drift around the gravity vector by measuring the Earth's magnetic field direction, providing the linearised attitude heading error $\delta\theta_z$.

The optimal Kalman gain matrix $K \in \mathbb{R}^{15 \times n}$ is computed as:

$$K = P_{\{k|k-1\}} \cdot H^T \cdot (H \cdot P_{\{k|k-1\}} \cdot H^T + R)^{-1} \quad (3.7)$$

The optimal error state correction is then:

$$\delta\hat{x} = K \cdot (y - h(x_{\text{nom}})) \quad (3.8)$$

where y is the measurement vector, $h(\cdot)$ is the nonlinear measurement function, and $H = \partial h / \partial \delta x$ is its Jacobian with respect to the error state. The estimated error $\delta\hat{x}$ is injected back into the nominal state:

$$p \leftarrow p + \delta\hat{p} \quad (3.9a)$$

$$v \leftarrow v + \delta\hat{v} \quad (3.9b)$$

$$q \leftarrow q \otimes \exp(\frac{1}{2} \cdot \delta\hat{\theta}) \quad (3.9c)$$

$$b_a \leftarrow b_a + \delta\hat{b}_a \quad (3.9d)$$

$$b_\omega \leftarrow b_\omega + \delta\hat{b}_\omega \quad (3.9e)$$

Following error injection, the error state is mathematically reset to zero: $\delta x \leftarrow 0$, and the covariance is updated via the Joseph form for numerical stability:

$$P \leftarrow (I - K \cdot H) \cdot P_{\{k|k-1\}} \cdot (I - K \cdot H)^T + K \cdot R \cdot K^T \quad (3.10)$$

3.5 Gated Recurrent Unit (GRU v3) Learned Navigation Model

Classical linear controllers—such as cascaded PID loops—perform adequately in calm, structured air environments, but struggle severely when navigating high-speed trajectories through urban corridors where aerodynamic drag, rotor downwash, building wake vortices, and actuator delays induce non-negligible tracking errors. The GRU v3 Learned Navigation Model acts as an intelligent predictive guidance layer that continuously adapts the quadrotor's velocity commands based on its temporal flight history.

3.5.1 Temporal Feature Buffer Formulation (101×24)

At each simulation step, the navigation system captures a 24-dimensional feature vector encoding the UAV's complete kinematic state and trajectory context, then appends it to a historical sliding-window buffer spanning $T = 101$ consecutive timesteps (approximately 2.0 seconds of flight history at 50 Hz). Table 3.2 details the precise mathematical ordering and physical semantics of these 24 features:

Table 3.2: 24-Dimensional Feature Vector — Ordering, Categories, and Physical Semantics

Feature Indices	Category	Variable Names	Physical Description
0 – 2	Relative Position	dx, dy, dz	Vector from current ESKF position to target waypoint (m)
3 – 5	ESKF Velocity	vx, vy, vz	Filtered 3-D velocity vector in world frame (m s^{-1})
6 – 9	Attitude Quaternion	qw, qx, qy, qz	Estimated orientation quaternion on S^3 unit sphere
10 – 12	Angular Velocity	$\omega_x, \omega_y, \omega_z$	Filtered body-frame rotational rates (rad s^{-1})
13 – 15	Linear Acceleration	ax, ay, az	Body-frame linear acceleration, gravity removed (m s^{-2})
16 – 19	Tracking Metrics	dist, CTE, ATE, speed	Target distance, cross-track and along-track errors, speed
20 – 23	Control History	cmd_vx, cmd_vy, cmd_vz, ψ	Preceding velocity commands and yaw rate dispatched ($\text{m s}^{-1}, \text{rad s}^{-1}$)

3.5.2 Gated Recurrent Unit Architecture and Gating Mechanism

Standard Recurrent Neural Networks (RNNs) suffer from vanishing and exploding gradients when processing long sequences, causing them to forget relevant flight history beyond

a few timesteps. While Long Short-Term Memory (LSTM) networks resolve this with three gates, the Gated Recurrent Unit (GRU) achieves comparable representational power with only two gates—the Reset Gate and the Update Gate—reducing the parameter count by approximately 25% and enabling ultra-low-latency inference. The GRU internal recurrence mechanics at timestep t are defined as:

$$r_t = \sigma(W_r \cdot x_t + U_r \cdot h_{t-1} + b_r) \quad (3.11) \text{ Reset Gate}$$

$$z_t = \sigma(W_z \cdot x_t + U_z \cdot h_{t-1} + b_z) \quad (3.12) \text{ Update Gate}$$

$$\tilde{h}_t = \tanh(W_h \cdot x_t + U_h \cdot (r_t \odot h_{t-1}) + b_h) \quad (3.13) \text{ Candidate Hidden State}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (3.14) \text{ Hidden State Update}$$

where $\sigma(\cdot)$ denotes the sigmoid activation function, $\odot$ is the element-wise Hadamard product, and $h_t \in \mathbb{R}^{128}$ encodes the accumulated spatio-temporal aerodynamic state of the quadrotor. The final hidden state h_{101} is projected through a fully connected multi-layer perceptron ($\text{Linear}(128, 64) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.1) \rightarrow \text{Linear}(64, 3)$) to produce the three-dimensional velocity correction vector:

$$\Delta v = [\Delta v_x, \Delta v_y, \Delta v_z]^T = \text{MLP}(h_{101}) \quad (3.15)$$

This correction is summed with the nominal kinematic waypoint guidance velocity: $v_{\text{guidance}} = v_{\text{nominal}} + \Delta v$, effectively compensating for cross-track drift, momentum overshoot, and wind-induced track deviation.

3.6 PULP-DroNet v3 Deep Vision Obstacle Avoidance Model

While the 15-state ESKF and GRU model ensure precision following of pre-planned global waypoints, real-world urban flight inevitably encounters unmapped, dynamic, or sudden obstacles—construction scaffolding, cranes, fallen trees, utility cables, and other aerial vehicles. The PULP-DroNet v3 deep vision model serves as the UAV's reactive obstacle avoidance cortex, inspired by the DroNet architecture tailored for micro-aerial vehicles operating on Parallel Ultra-Low-Power (PULP) embedded computing.

3.6.1 Monocular Synthetic Sensor Camera Pipeline

The simulation pipeline renders a dedicated forward-facing monocular sensor viewport rigidly mounted to the UAV's front airframe. The sensor camera operates at an effective resolution of 200×200 pixels in eight-bit grayscale format with a 75-degree horizontal Field of View (FoV). Grayscale conversion suppresses distracting colour variations while preserving structural edge contrast critical for obstacle detection. Pixel intensities are normalised to the floating-point range $[0.0, 1.0]$ and packaged into a PyTorch input tensor of shape $[1, 1, 200, 200]$.

3.6.2 Deep Residual Architecture (ResBlock-1.0)

PULP-DroNet v3 utilises a lightweight Residual Convolutional Neural Network (CNN) designed to extract multi-scale spatial features without the prohibitive computational burden of full-scale networks such as ResNet-50. The architecture consists of an initial strided convolution stem followed by three cascading Residual Blocks with identity and projection skip connections. Table 3.3 summarises the complete layer-wise topology:

Table 3.3: PULP-DroNet v3 Layer-Wise Architecture (Input: $1 \times 200 \times 200$)

Stage	Kernel / Stride	Output Shape	Activation	Functionality
Input	—	$1 \times 200 \times 200$	—	Normalised monocular grayscale feed
Conv Stem	$5 \times 5, s = 2 +$ MaxPool 2×2	$32 \times 50 \times 50$	ReLU	Low-level edge and gradient features
ResBlock 1	$3 \times 3, s = 1 (\times 2)$	$32 \times 50 \times 50$	ReLU + Add	Surface texture and boundary details
ResBlock 2	$3 \times 3, s = 2 (\times 2)$	$64 \times 25 \times 25$	ReLU + Conv Skip	Spatial downsampling, obstacle profiles
ResBlock 3	$3 \times 3, s = 2 (\times 2)$	$128 \times 13 \times 13$	ReLU + Conv Skip	High-level corridor geometry
Dual FC Head	FC(128) $\rightarrow$ two heads	α : scalar P_coll: scalar	Linear / Sigmoid	Steering angle and collision risk

3.6.3 Dual-Head Output and Multi-Task Loss Formulation

A defining strength of DroNet is its unified dual-head prediction architecture. Unlike standard object detectors that require complex bounding-box post-processing pipelines, DroNet directly outputs two compact, immediately actionable flight navigation variables:

Steering Angle Head $\alpha \in [-1.0, +1.0]$: A continuous regression output predicting normalised lateral steering offset. $\alpha = 0$ corresponds to straight flight; negative values command left evasion; positive values command right evasion relative to the current heading.

Collision Probability Head $P_{\text{coll}} \in [0.0, 1.0]$: A sigmoidal probability output predicting the likelihood of impending collision within a 15-metre forward stopping horizon. $P_{\text{coll}} < 0.3$ indicates a clear corridor; $P_{\text{coll}} > 0.7$ indicates immediate collision danger.

The model is trained with a multi-task loss function:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MSE}}(\alpha, \hat{\alpha}) + \lambda \cdot \mathcal{L}_{\text{BCE}}(P_{\text{coll}}, \hat{P}_{\text{coll}}) \quad (3.16)$$

where $\mathcal{L}_{\text{MSE}}$ is Mean Squared Error applied to the continuous steering angle regression, $\mathcal{L}_{\text{BCE}}$ is Binary Cross-Entropy applied to the binary collision label, and $\lambda = 1.0$ is a balancing hyperparameter that ensures equal gradient propagation across both output heads during training.

3.7 Model Interconnection and Hybrid Control Arbitration

A central innovation of this project is the hierarchical interconnection and dynamic arbitration between the 15-state ESKF, the GRU learned navigation model, and the PULP-DroNet visual obstacle avoidance network. The following description outlines the complete inter-model control topology:

At each control cycle, the ESKF outputs the filtered kinematic state $[p, v, q, b_a, b_\omega]$. This state is used to construct the 24-dimensional feature vector, which—together with its 101-step history—is transmitted asynchronously to the GRU service (Port 8765). The GRU service returns the velocity correction Δv , which is added to the nominal waypoint guidance velocity to form the planned command v_{GRU} . Concurrently, the forward sensor camera renders a 200×200 grayscale frame that is transmitted to the DroNet service (Port 8766), which returns steering

angle α and collision probability P_{coll} . The Hybrid Control Arbitrator then blends these two signals according to:

$$v_{\text{cmd}} = (1 - w_{\text{avoid}}) \cdot v_{\text{GRU}} + w_{\text{avoid}} \cdot v_{\text{avoid}}(\alpha) \quad (3.17)$$

where the avoidance weight $w_{\text{avoid}} \in [0, 1]$ transitions continuously through three operating regimes based on the instantaneous collision probability P_{coll} :

Regime 1 — Free Corridor Flight ($P_{\text{coll}} < 0.30$): The avoidance weight $w_{\text{avoid}} = 0$. The quadrotor executes nominal waypoint navigation governed entirely by the GRU model: $v_{\text{cmd}} = v_{\text{GRU}}$.

Regime 2 — Active Blended Avoidance ($0.30 \leq P_{\text{coll}} < 0.75$): A proximate obstacle has been detected. The avoidance weight scales smoothly as $w_{\text{avoid}} = (P_{\text{coll}} - 0.30) / (0.75 - 0.30)$. The guidance vector is rotated laterally by a component proportional to DroNet's steering angle: $v_{\text{avoid}} = v_{\text{nominal}} + k_{\text{steer}} \cdot \alpha \cdot \hat{n}_{\text{lat}}$.

Regime 3 — Emergency Critical Braking ($P_{\text{coll}} \geq 0.75$): The obstacle is inside the emergency safety bubble. The arbitrator overrides all waypoint guidance ($w_{\text{avoid}} = 1.0$): forward velocity is actively braked to zero, a maximum lateral evasion thrust is commanded in direction $\text{sign}(\alpha)$, and an emergency vertical climb of $\Delta v_z = +2.5 \text{ m s}^{-1}$ is initiated until P_{coll} falls below the recovery threshold of 0.30.

3.8 Project Execution and Deployment Guide

To ensure complete reproducibility and straightforward deployment on any workstation, the project is containerised into standardised Node.js and Python microservices with unified launch scripts. The following subsections describe all prerequisites and step-by-step procedures.

3.8.1 Hardware and Software Prerequisites

Operating System: Windows 10 / 11, Ubuntu 22.04 LTS, or macOS (Apple Silicon or Intel).

Runtime Environments: Node.js version 18.0 or higher and Python version 3.10 or 3.11.

Python Libraries: PyTorch ($\geq 2.2.0$), FastAPI ($\geq 0.115.0$), Uvicorn ($\geq 0.30.0$), NumPy ($\geq 1.26.0$), Pillow ($\geq 10.0.0$), Pydantic ($\geq 2.0.0$).

Node.js Libraries: Three.js ($\geq 0.170.0$), Vite ($\geq 6.0.0$).

GPU Acceleration: A dedicated NVIDIA GPU with CUDA 12.x is strongly recommended. CPU inference fallback is automatically supported.

Web Browser: Google Chrome (v120+), Microsoft Edge (v120+), or Mozilla Firefox (v121+) with WebGL 2 support.

3.8.2 Repository Structure

src/: Core Three.js simulation modules (DronePhysics.js, ESKF.js, RouteController.js, CityGenerator.js, WeatherSystem.js).

public/materials/: Photorealistic PBR texture maps and binary glTF (.glb) urban models.

GRU MODEL/: PyTorch GRU v3 implementation (model_runtime.py, server_fastapi.py) with pre-trained weights.

dronet_service/: PULP-DroNet v3 PyTorch model (dronet_model.py, server.py) and ResBlock-1.0 weights.

run_simulation.bat: One-click Windows launcher that configures environments and starts all services automatically.

requirements.txt: Python dependency specification file for all AI model services.

3.8.3 Method 1: One-Click Automated Launch (Windows)

On any Windows workstation, the complete simulation stack—three-dimensional city, GRU navigation model, and DroNet obstacle avoidance—can be launched in a single step:

Step 1: Clone or download the repository from <https://github.com/sarialbasha3-pixel/drone-city-simulation>.

Step 2: Open the project root folder in Windows Explorer.

Step 3: Double-click run_simulation.bat, or execute .\run_simulation.bat in PowerShell.

Automatic: The batch launcher checks for Node.js modules (running npm install if absent), creates a Python virtual environment (.venv) and installs requirements.txt if needed, launches both FastAPI microservices in titled background consoles (Ports 8765 and 8766),

starts the Vite development server on port 3000, and automatically opens the simulation in the default browser.

3.8.4 Method 2: Manual Multi-Terminal Startup (Cross-Platform)

For Linux, macOS, or advanced debugging, the services are initialised manually across three terminal sessions:

```
Step 1 – Install dependencies (once only):  
    npm install  
    python -m venv .venv  
    .venv/Scripts/pip install -r requirements.txt  
  
Step 2 – Terminal A: Launch GRU Navigation AI:  
    python -m uvicorn server_fastapi:app --app-dir "GRU MODEL" --host  
127.0.0.1 --port 8765  
  
Step 3 – Terminal B: Launch PULP-DroNet Vision AI:  
    python -m uvicorn server:app --app-dir dronet_service --host 127.0.0.1 --  
port 8766  
  
Step 4 – Terminal C: Launch Three.js Simulation:  
    npx vite --host --port 3000  
  
Step 5 – Open http://localhost:3000 in Chrome / Edge / Firefox.
```

3.8.5 Service Health Verification and Telemetry Validation

Once all three services are active, the analyst can verify operational status using the built-in REST health endpoints before beginning flight experiments:

GRU Service: Visit `http://127.0.0.1:8765/health` → Returns `{"ok": true, "model": "Project Ghost GRU V3", "input_shape": [101, 24]}`.

DroNet Service: Visit `http://127.0.0.1:8766/health` → Returns `{"ok": true, "model": "PULP-DroNet v3 (ResBlock-1.0)", "device": "cuda" or "cpu"}`.

Simulation HUD: The top-right telemetry overlay in the Three.js viewport displays live green 'AI CONNECTED' status indicators for both models.

3.9 Chapter Summary

This chapter has established the comprehensive architectural and methodological foundation of the autonomous urban drone navigation system. A modular, decoupled microservices framework was formulated that unites photorealistic three-dimensional urban simulation with real-time state estimation and multi-tiered artificial intelligence. The specific contributions of this chapter are summarised as follows:

Photorealistic Simulation Environment: Leverages procedural generation, PBR Metallic-Roughness materials, and GPU-instanced three-dimensional assets to create visually realistic, performance-optimised urban flight corridors.

15-State Error-State Extended Kalman Filter (ESKF): Provides singularity-free, minimal-parameter state estimation on the $SO(3)$ Lie group by propagating nominal kinematics at 100 Hz and correcting 15 error states using asynchronous multi-sensor updates from GPS, barometer, and magnetometer.

GRU v3 Learned Navigation Model: Ingests a 101×24 historical feature tensor through a two-gate recurrent architecture to predict smooth velocity adjustments Δv that compensate for dynamic aerodynamic disturbances, momentum overshoot, and path curvature.

PULP-DroNet v3 Deep Vision Model: Processes monocular 200×200 grayscale sensor feeds through three cascading Residual Convolutional Blocks, outputting continuous steering angles and sigmoidal collision hazard probabilities in fewer than 12 ms.

Hybrid Control Arbitration: Harmonises global route guidance and reactive visual avoidance through a mathematically grounded arbitration matrix, transitioning continuously between free flight, active blended avoidance, and emergency critical braking according to the instantaneous DroNet collision probability P_{coll} .

With the system architecture and theoretical methodology thoroughly formalised, Chapter Four presents the experimental setup, trajectory tracking performance metrics, obstacle avoidance benchmarks, and comparative performance analyses conducted under varied weather conditions and synthetic sensor noise profiles.